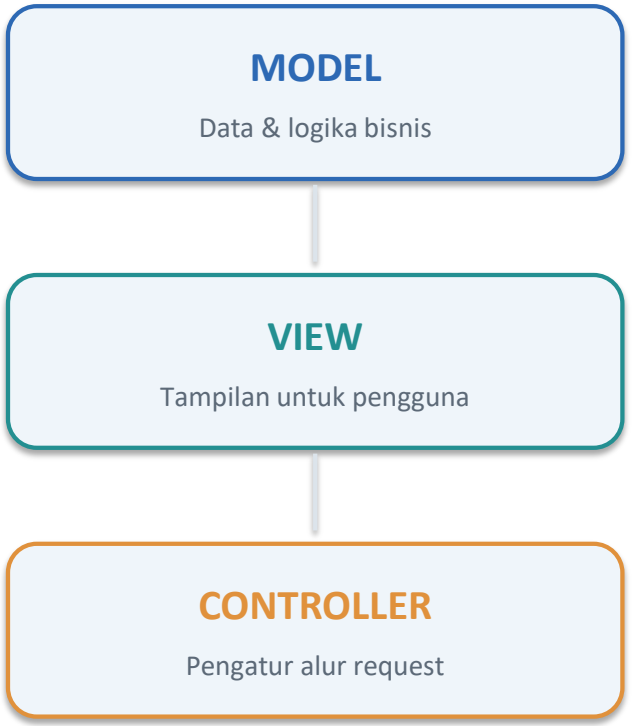
MODEL MVC

Model – View – Controller

Memahami pemisahan data, tampilan, dan alur kontrol dalam aplikasi web



Mengapa Struktur Kode Penting?

Ketika semua dicampur

Satu file berisi query SQL, aturan bisnis, dan HTML. Perubahan kecil dapat memengaruhi banyak bagian dan kode makin sulit dirawat.

Ketika tanggung jawab dipisah

Setiap bagian punya fokus. Data dapat diuji terpisah dari tampilan; desain dapat berubah tanpa harus mengubah aturan bisnis.

Contoh masalah tanpa pemisahan

```
<?php
// Query database + aturan bisnis + HTML bercampur dalam satu file
$result = $db->query('SELECT nama FROM mahasiswa');
echo '<h1>Daftar Mahasiswa</h1>'; // tampilan dibangun di tempat yang sama
```

MVC menawarkan cara mengorganisasi aplikasi agar perubahan lebih terarah.

Sejarah Singkat dan Konsep Dasar

Akhir 1970-an

Trygve Reenskaug merumuskan MVC saat bekerja di Xerox PARC. Pola ini digunakan pada antarmuka grafis desktop, termasuk lingkungan Smalltalk.

Akhir 1990-an–awal 2000-an

Prinsip MVC diadaptasi dalam pengembangan aplikasi web dan menjadi pola penting dalam berbagai framework.

Penggunaan modern

Contoh framework yang mengadopsi MVC atau variasinya: Ruby on Rails, Laravel, ASP.NET Core MVC, Spring MVC, serta Django dengan MTV.

Konsep inti: Separation of Concerns

Pemisahan tanggung jawab berarti setiap komponen berfokus pada satu jenis pekerjaan, bukan mengambil alih tugas komponen lain.

Analogi sederhana: petugas data mengurus informasi, desainer mengurus penyajian, dan koordinator mengatur proses.

Prinsip yang Mendukung MVC

Separation of concerns

Data, tampilan, dan alur kontrol ditempatkan pada komponen berbeda.

Reusability

Model yang sama dapat menyuplai data ke lebih dari satu View, misalnya halaman web dan API.

Maintainability

Perubahan desain tidak harus mengubah logika bisnis, dan sebaliknya.

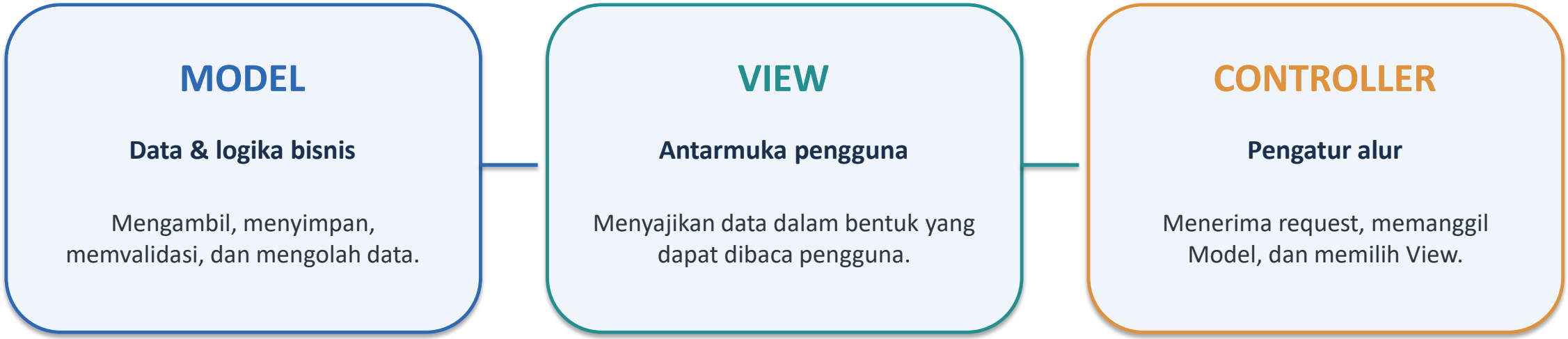
Parallel development

Pengembang back-end dan perancang front-end dapat bekerja secara paralel dengan kesepakatan antarmuka yang jelas.

Pemisahan bukan sekadar membuat banyak file; yang utama adalah menjaga batas tanggung jawab.

Gambaran Umum Tiga Komponen MVC

MVC membagi aplikasi menjadi tiga peran utama:



Ingat: Model tidak mengatur HTML, View tidak mengambil data dari database secara langsung, dan Controller tidak seharusnya menampung logika bisnis yang rumit.

Hubungan MVC



- Bagian yang bertugas menyajikan kepada pengguna.
- Menampilkan data dalam bentuk User Interface (GUI).
- Fokus pada apa yang dilihat dan langsung dengan user.

- Otak yang menghubungkan Model dan View.
- Menerima request (permintaan) dari pengguna (melalui View).
- Menentukan data apa yang harus diambil dari Model dan bagaimana data tersebut ditampilkan kembali.

- Bagian yang mengelola dan langsung dengan database.
- Bertugas untuk menyiapkan, mengatur, dan mengorganis aikan data.
- Pusat kebenaran (source of truth) dari sebuah aplikasi.

Gemini Notebook

Memahami MVC Lewat Analogi

Analogi Restoran

Konsep abstrak MVC paling mudah dipahami dengan membayangkan sebuah restoran. Dalam sistem ini, ada pelanggan yang memesan, pelayan yang mencatat dan mengantar, serta koki yang memasak di dapur.



Gemini Notebook



Pelanggan = View

Pelanggan berinteraksi dengan buku menu dan meja makan. Ini sama dengan user yang mengakses antarmuka website.



Pelayan = Controller

Menerima pesanan pelanggan dan meneruskannya ke dapur. Mewakili Controller yang merespons request dari View.



Koki = Model

Menerima catatan dari pelayan, mencari bahan di kulkas (Database), dan mengolah makanan (Data).

Gemini Notebook

Komponen 1 — Model

Model mengelola data dan aturan bisnis aplikasi.

- **Berhubungan dengan data**
  Mengambil atau menyimpan informasi melalui database.
- **Menjalankan aturan**
  Memvalidasi data dan menjalankan perhitungan/aturan aplikasi.
- **Tidak mengatur presentasi**
  Model tidak perlu mengetahui warna tombol atau susunan HTML.
- **Contoh**
  Mengambil daftar mahasiswa, menyimpan pesanan, menghitung total belanja.

Contoh pemikiran Model

“Data mahasiswa apa yang tersedia?”

“Apakah input memenuhi aturan?”

“Berapa total belanja yang benar?”

Bentuk umum

Kelas/objek yang mewakili data atau menyediakan operasi terhadap data.

Model berfokus pada apa yang diketahui dan aturan apa yang berlaku pada data.

Komponen 2 — View

View bertugas menyajikan data kepada pengguna.

- **Bentuk umum**
  HTML, CSS, dan sedikit logika presentasi.
- **Menerima data**
  Data disiapkan oleh Controller lalu ditampilkan oleh View.
- **Tidak mengakses database langsung**
  View tidak seharusnya menjalankan query SQL.
- **Contoh**
  Tabel daftar mahasiswa, form input, halaman login.

Pertanyaan yang dijawab View

“Bagaimana daftar mahasiswa ditampilkan?”

“Kolom apa yang terlihat?”

“Bagaimana form ditata agar mudah dipakai?”

Contoh file

views/mahasiswa/index.php

Pada Laravel dapat berupa template Blade (.blade.php).

View berfokus pada bagaimana informasi disajikan, bukan dari mana data diperoleh.

Komponen 3 — Controller

Controller menjadi perantara antara request pengguna, Model, dan View.

- **Menerima request**
  Misalnya pengguna membuka halaman atau mengirim form.
- **Memilih tindakan**
  Menentukan Model/method yang perlu dipanggil.
- **Meneruskan hasil**
  Mengirim data yang diperlukan kepada View.
- **Tetap tipis**
  Logika bisnis kompleks sebaiknya berada di Model, bukan menumpuk di Controller.

Pertanyaan yang dijawab Controller

“Request ini harus melakukan apa?”

“Model mana yang dibutuhkan?”

“View mana yang sesuai?”

Contoh tindakan

Membaca input form, meminta Model menyimpan data, lalu mengarahkan pengguna ke halaman hasil.

Controller mengatur urutan kerja, bukan menjadi tempat semua pekerjaan dilakukan.

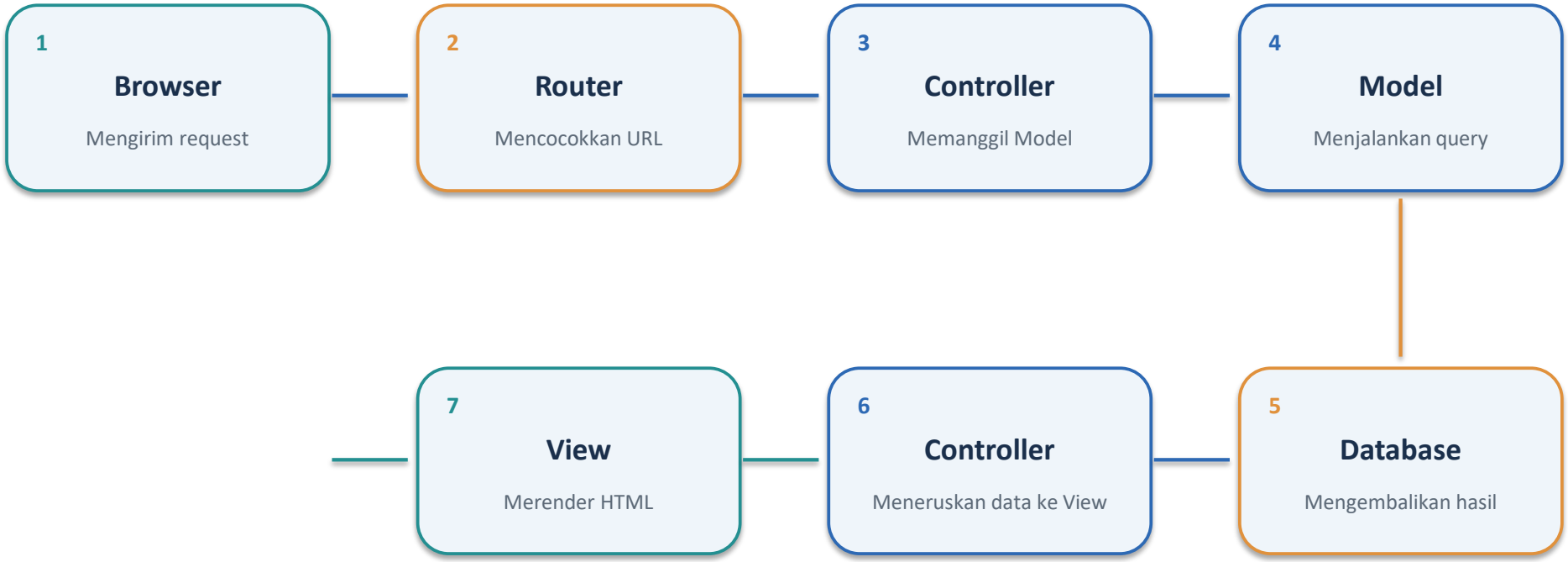
Batas Tanggung Jawab: Boleh dan Tidak Boleh

Komponen	Tanggung jawab utama	Hindari
Model	Data, query, validasi, aturan bisnis	Menyusun HTML atau mengatur tampilan
View	Menampilkan data yang disediakan	Menjalankan query database langsung
Controller	Menerima request, memanggil Model, memilih View	Menampung logika bisnis yang rumit

Tes cepat: jika tugasnya menjawab “bagaimana tampilan?”, pikirkan View. Jika menjawab “apa aturan datanya?”, pikirkan Model. Jika menjawab “apa langkah berikutnya?”, pikirkan Controller.

Alur Kerja Request–Response pada MVC

Contoh: pengguna membuka URL /mahasiswa untuk melihat daftar mahasiswa.



Hasil akhirnya: server mengirim response berupa HTML ke browser. Browser menampilkan halaman kepada pengguna.

Peran Router dalam Aplikasi Web

Apa yang dilakukan Router?

Router mencocokkan URL dan metode request dengan tindakan/Controller yang sesuai. Contoh: /mahasiswa diarahkan ke MahasiswaController.

Apakah Router termasuk MVC?

Router bukan salah satu dari tiga komponen inti MVC. Namun, Router hampir selalu ada pada framework web untuk menentukan ke mana request diarahkan.

Skema sederhana

```
Browser: GET /mahasiswa  →  Router  →  MahasiswaController@index  →  Model  →  View
```

Routing membantu aplikasi berkembang tanpa menumpuk banyak kondisi URL dalam satu file.

Implementasi PHP Native: Struktur Folder

Contoh kasus: menampilkan daftar mahasiswa dari database tanpa framework.

```
project/
├── index.php
├── config/
│   └── database.php
├── models/
│   └── Mahasiswa.php
├── controllers/
│   └── MahasiswaController.php
└── views/
    └── mahasiswa/
        └── index.php
```

config/database.php

Konfigurasi koneksi database; menghasilkan objek PDO.

models/Mahasiswa.php

Mengambil data mahasiswa dari database.

controllers/MahasiswaController.php

Memanggil Model dan menyiapkan data untuk View.

views/mahasiswa/index.php

Menyajikan daftar mahasiswa dalam bentuk tabel HTML.

Kode Model — Mengambil Data

File: models/Mahasiswa.php

```php
<?php
class Mahasiswa {
    private $db;

    public function __construct($db) {
        $this->db = $db;
    }

    public function semua() {
        $stmt = $this->db->query(
            'SELECT nim, nama, prodi FROM mahasiswa'
        );
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }
}
```

Apa yang dilakukan?

Menyimpan koneksi PDO dan menyediakan method semua() untuk mengambil data.

Apa yang dikembalikan?

Array asosiatif berisi kolom nim, nama, dan prodi untuk setiap baris.

Model tidak menentukan apakah data ditampilkan sebagai tabel atau kartu.

Kode Controller — Menghubungkan Model dan View

File: controllers/MahasiswaController.php

```php
<?php
require_once 'models/Mahasiswa.php';

class MahasiswaController {
    private $model;

    public function __construct($db) {
        $this->model = new Mahasiswa($db);
    }

    public function index() {
        $daftar = $this->model->semua();
        require 'views/mahasiswa/index.php';
    }
}
```

Langkah 1

Controller membuat objek Mahasiswa dan memberinya koneksi database.

Langkah 2

Method index() meminta data lewat semua(), lalu memuat View.

Variabel $daftar tersedia saat file View dimuat dengan require.

Kode View — Menampilkan Data

File: views/mahasiswa/index.php

```
<h1>Daftar Mahasiswa</h1>
<table border="1">
  <tr><th>NIM</th><th>Nama</th><th>Prodi</th></tr>
  <?php foreach ($daftar as $m): ?>
    <tr>
      <td><?= htmlspecialchars($m['nim']) ?></td>
      <td><?= htmlspecialchars($m['nama']) ?></td>
      <td><?= htmlspecialchars($m['prodi']) ?></td>
    </tr>
  <?php endforeach; ?>
</table>
```

Perulangan foreach

Mengulang setiap baris mahasiswa dan membentuk satu baris tabel.

htmlspecialchars()

Mengubah karakter khusus agar input tidak ditafsirkan sebagai HTML/script saat ditampilkan.

View hanya menyajikan data yang telah disiapkan; tidak melakukan query SQL.

Front Controller — Pintu Masuk Aplikasi

File: index.php — contoh router sederhana

```php
<?php
require 'config/database.php'; // menghasilkan objek PDO: $db
require 'controllers/MahasiswaController.php';

$halaman = $_GET['page'] ?? 'mahasiswa';

if ($halaman === 'mahasiswa') {
    $controller = new MahasiswaController($db);
    $controller->index();
} else {
    http_response_code(404);
    echo 'Halaman tidak ditemukan';
}
```

Pintu masuk

Request aplikasi diarahkan melalui satu file awal.

Parameter page

Jika page tidak diberikan, aplikasi memilih mahasiswa sebagai nilai default.

Catatan: router ini disederhanakan untuk pembelajaran; framework menyediakan routing lebih lengkap.

Menyatukan Kode: Apa yang Terjadi Saat Dibuka?

1. Browser	Pengguna membuka index.php?page=mahasiswa.
2. Front controller	index.php memuat konfigurasi database dan controller.
3. Controller	MahasiswaController dibuat lalu method index() dipanggil.
4. Model	Method semua() mengambil data dari tabel mahasiswa.
5. View	Variabel $daftar digunakan untuk membuat tabel HTML.
6. Response	HTML dikirim ke browser dan ditampilkan.

Kunci pemahaman: data mengalir dari Model → Controller → View; request masuk melalui routing/controller.

Catatan Keamanan pada Contoh

Mengurangi risiko XSS

View menggunakan htmlspecialchars() untuk menampilkan nilai data sebagai teks, bukan membiarkannya diperlakukan sebagai markup HTML/script.

Mengurangi risiko SQL injection

PDO mendukung prepared statement. Untuk query yang menerima input pengguna, gunakan parameter terikat, bukan menggabungkan input langsung ke string SQL.

Penting untuk mahasiswa

- Contoh query pada Model di materi tidak menerima input pengguna secara langsung.
- Jika menambahkan pencarian berdasarkan NIM, gunakan prepared statement dengan parameter.
- Keamanan bukan hanya urusan satu file; validasi input, otorisasi, dan penanganan error juga perlu dirancang.

MVC membantu mengorganisasi kode, tetapi tidak otomatis membuat aplikasi aman.

Kelebihan dan Kekurangan MVC

KELEBIHAN

- Kode lebih rapi dan mudah dicari/dirawat.
- Front-end dan back-end dapat bekerja paralel.
- Model dan Controller dapat diuji secara terpisah.
- Satu Model dapat digunakan oleh beberapa View.
- Pola yang banyak dipakai dalam framework.

KEKURANGAN

- Jumlah file dan folder bertambah.
- Perlu kesepakatan penamaan dan struktur tim.
- Pengujian otomatis untuk View bisa lebih sulit.
- Batas peran bisa kabur jika logika bocor ke Controller.
- Kurva belajar awal lebih curam; berlebihan untuk halaman statis sangat sederhana.

Kesimpulan: MVC sangat berguna saat aplikasi mulai berkembang, tetapi tetap perlu diterapkan secara proporsional.

Framework yang Menggunakan MVC atau Variasinya

Laravel • PHP

Eloquent ORM untuk Model; Blade untuk View; Controller umumnya di app/Http/Controllers.

CodeIgniter • PHP

Framework relatif ringan dan sering digunakan untuk mempelajari MVC.

Django • Python

Menggunakan pola MTV (Model–Template–View), yang secara konsep sebanding dengan MVC.

Ruby on Rails • Ruby

Mempopulerkan pendekatan convention over configuration.

Spring MVC • Java

Digunakan pada berbagai aplikasi berbasis Java, termasuk lingkungan enterprise.

ASP.NET Core MVC • C#

Terintegrasi dengan ekosistem .NET.

Nama folder dan istilah dapat berbeda; pahami tanggung jawabnya, bukan hanya nama filenya.

MVC dan Pola yang Berkaitan

MVC

Controller mengatur alur request, memanggil Model, lalu memilih View untuk menampilkan hasil.

MVP

Presenter menggantikan Controller dan mengendalikan View secara lebih langsung; View cenderung lebih pasif.

MVVM

ViewModel menyediakan data/keadaan yang diikat ke View melalui data binding; umum pada sejumlah aplikasi modern.

MTV pada Django

Template berperan sebagai View, sedangkan komponen yang disebut View di Django memiliki peran yang mirip Controller pada MVC.

Pelajari MVC terlebih dahulu sebagai fondasi; setelah itu, variasi lebih mudah dibandingkan.

Kesalahan Pemahaman yang Perlu Dihindari

“Model adalah tampilan tabel.”	Tidak. Model mengelola data dan aturan; tampilan tabel adalah tugas View.
“View boleh query database karena lebih praktis.”	Tidak disarankan. View sebaiknya menerima data dari Controller.
“Semua logika harus ditulis di Controller.”	Tidak. Controller mengatur alur; aturan bisnis kompleks sebaiknya ditempatkan pada Model.
“Router adalah komponen keempat MVC.”	Router umum pada aplikasi web, tetapi bukan salah satu dari tiga komponen inti MVC.
“Memakai MVC otomatis membuat aplikasi aman.”	Tidak. MVC membantu struktur; keamanan tetap memerlukan validasi, query berparameter, output escaping, dan kontrol akses.